Getting started with Git/GitHub

ISN Methods Meeting 20-10-2026

Felix Klaassen

Name

 project_script.R

 project_script1.R

 project_script2.R

 project_script4.R

 project_script4bis.R

 project_script4bis_new.R

 project_script4bis_new_SP.R

 project_script_20191212.R

 project_script_20200122.R

 project_script_FINAL.R

 project_script_FINAL-FINAL.R

The goal of this meeting

- Introducing Git and GitHub for version control
  - What is it?
  - Why would you use it?
- Providing base level of knowledge/skills
  - How does it work
  - How do you use it?
- What I will not go into (unless there are questions)
  - How to deal with ignoring files (.gitignore)
  - More ‘advanced’ Git/GitHub functionalities
    - E.g. how to collaborate with others on a project (e.g., pull requests)
    - How to ‘edit’ your version history

Disclaimer

- I’m not an expert, just a basic user like you will be ☺

A source used for these slides: https://julia-pfarr.github.io/git-gitlab_tutorial/

What is it?

Git ≠ GitHub

Git	GitHub
A piece of software on your pc	An online repository (collaboration program)
Decentralized	Centralized
Local version control	Distributed version control

You can use Git without GitHub!

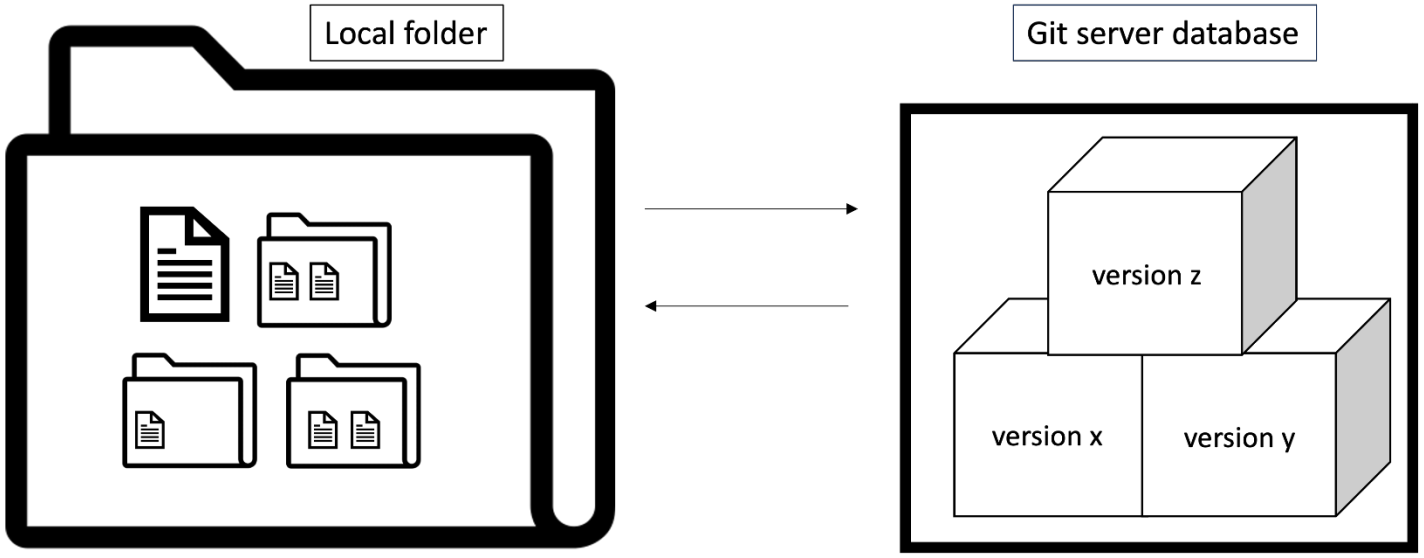
You can use GitHub without Git!

Why would you use it?

- Git
  - Track changes
  - Annotate each change with a message / description
  - Multiple streams of work (or version sets) independent from each other, while also being able to merge the work back together
  - Prevent bloating of your workflow with *v2, v3, v4, v4Final* versions
- GitHub
  - Code sharing!
  - No need to email around new version of files
  - Keeps all team members on the same page / working on the latest version

How does it work?

Git stores the different versions of your project for you (locally!)

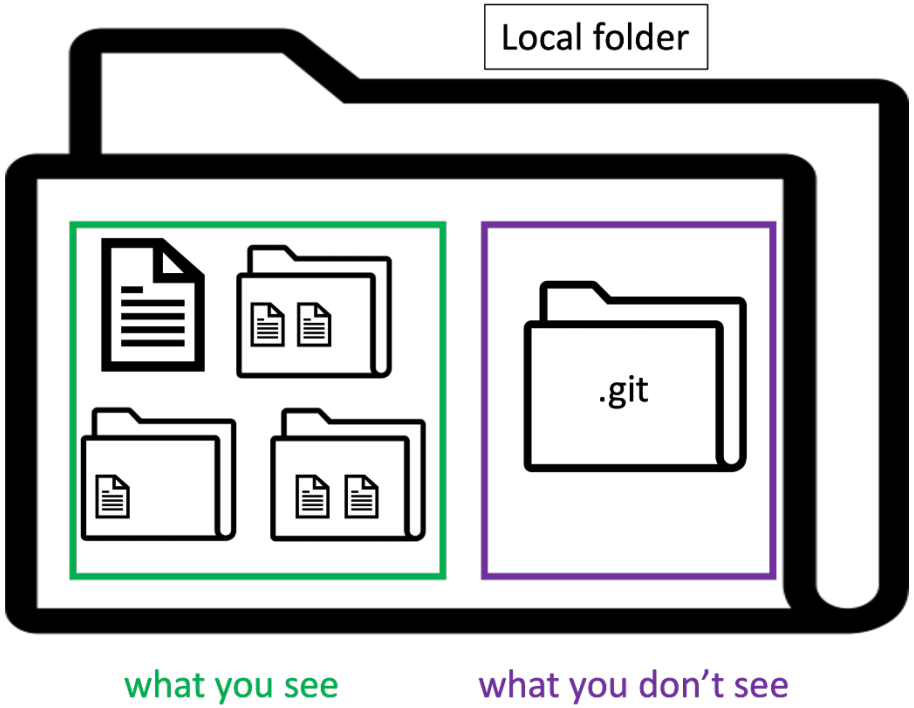


= what you see/ current version

= what is saved and can be looked at and recovered any time

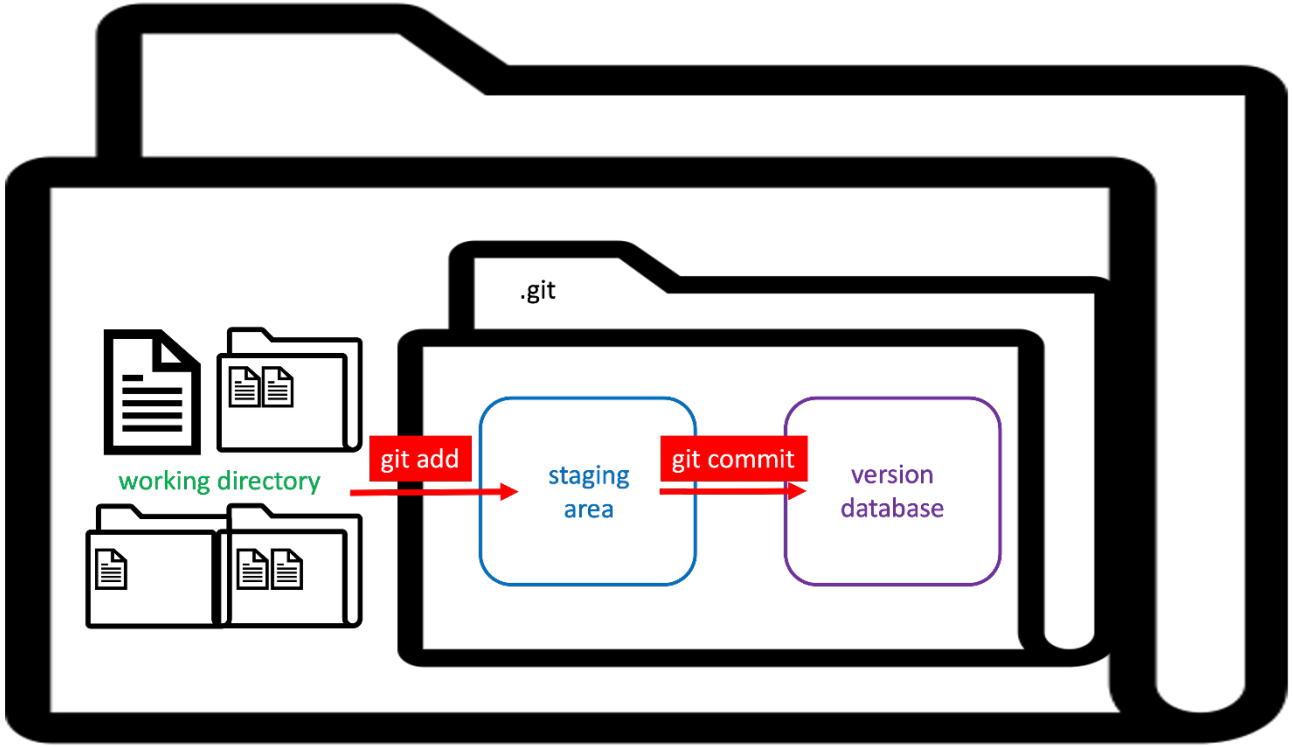
How does it work?

Git stores the different versions of your project for you (locally!)



How does it work?

Git stores the different versions of your project for you (locally!)



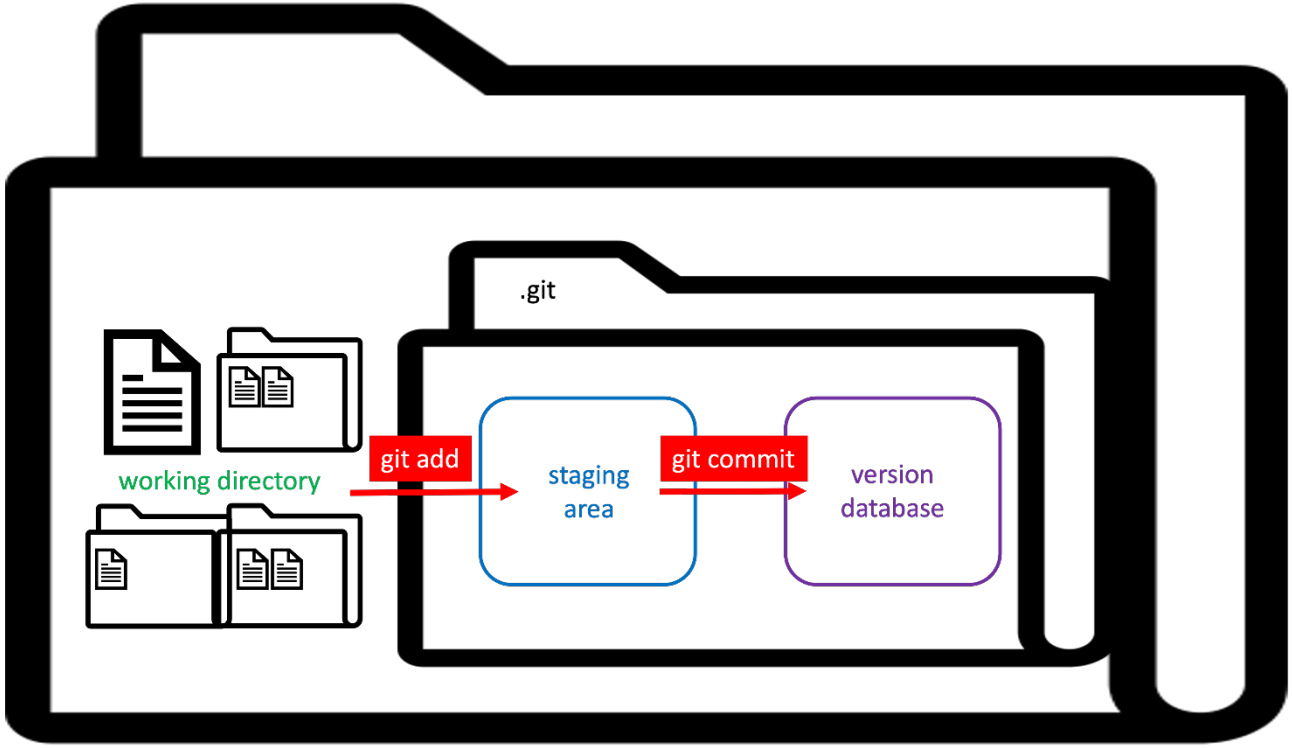
Basic workflow
(i.e., what you'll be doing 95% of the time)

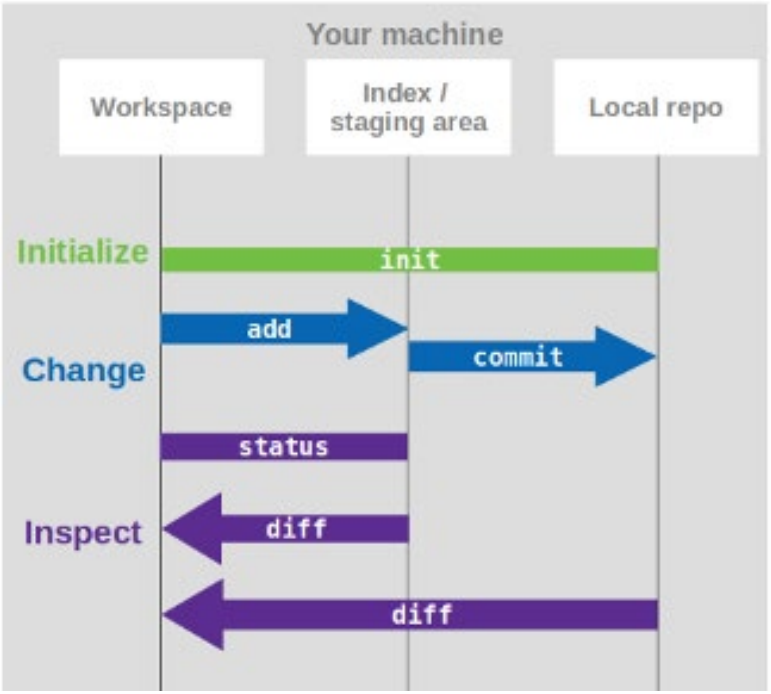
> Make some changes to your files
> '*git status*' check which files have changes
> '*git add*' changed files to the staging area
> '*git commit*' files from staging area to the db
> repeat
> ???
> Profit

Each 'git commit' requires a short message / description of the changes

How does it work?

Git stores the different versions of your project for you (locally!)





How do you use it?

- Download Git: https://git-scm.com/install/
- Communicate with Git via the Git Bash (i.e., a dedicated terminal)

Git command	What it does
git init	Initialize Git-based version control for a specified folder
git add <file>	Adds changes in your working directory (i.e., files) to the staging area
git commit <file>	Changes are commited, i.e., a snapshot of the current version of your files is saved in the version database (repository)
git status	Check the status of the files (if they contain changes, if they are in the staging area, etc.)
git diff	Lets you inspect differences between versions (have not used myself)
git log	Views the commit history (have not used myself)

Tip: instead of typing '*git add myFile.m*' for each file, you can also type '*git add .* ' to add all changes across files in one command (same for '*git commit .* ')

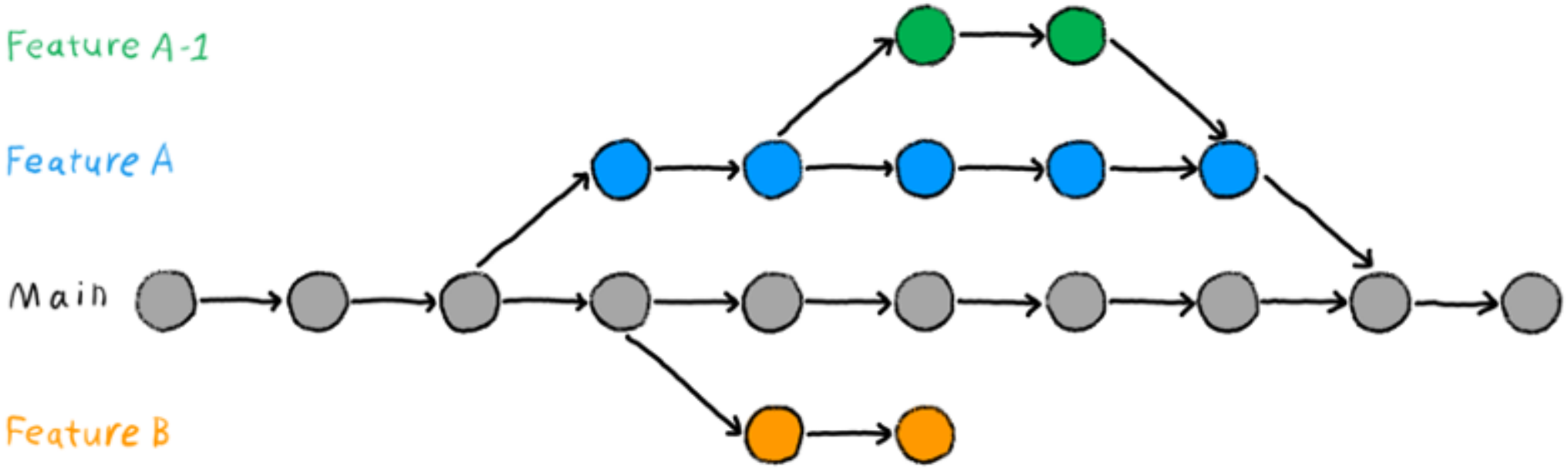
How do you use it?

- A tiny demo



How do you use it?

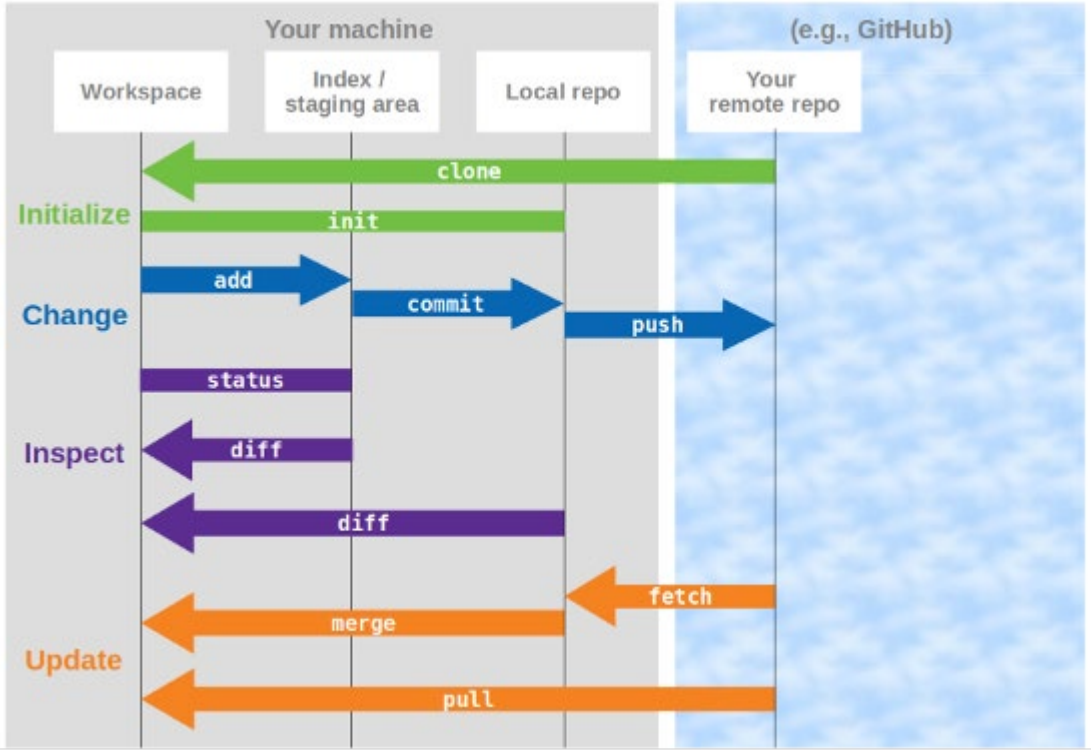
On branching and merging



Particularly useful in a collaborative context: Git**Hub**!

How does it work?

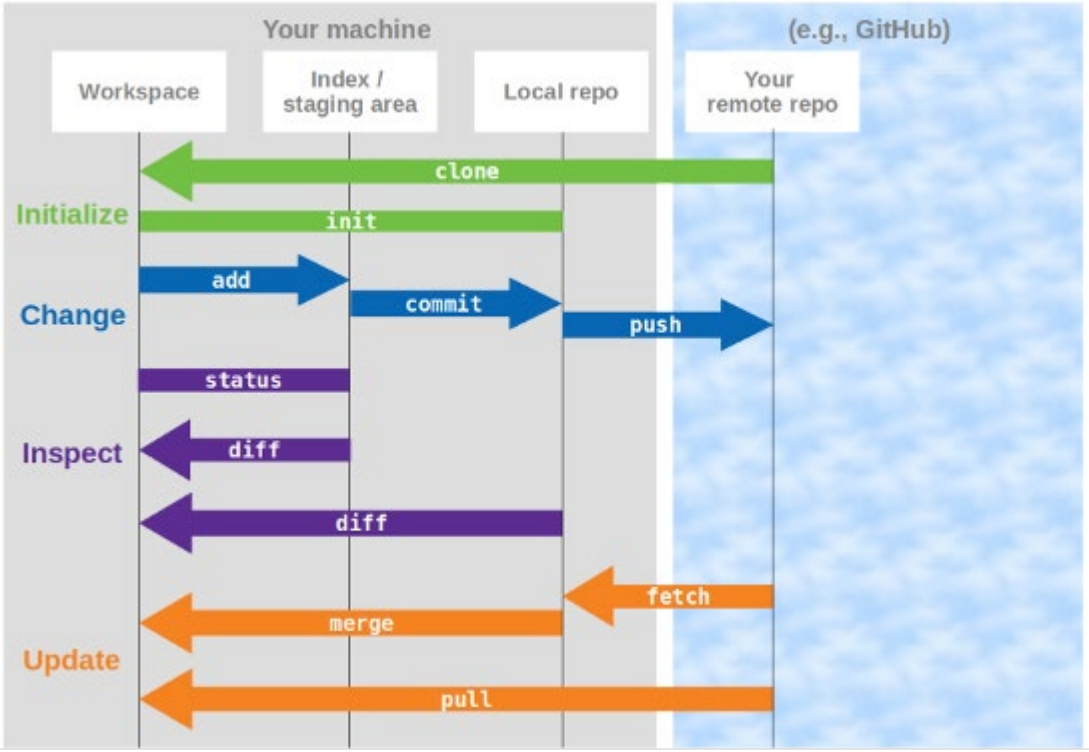
- Git**Hub** is a platform for collaborative code development
- Transferring **local** repositories to a **remote** repository (and vice versa)



Git command	What it does
git push	Synchronize your local and remote repository by 'pushing' changes from your local machine to the GitHub repo
git pull	Synchronize your local and remote repository by 'pulling' changes from the GitHub repo to your local machine
git clone	Create an exact copy of an existing GitHub repository on your local machine. Functions as a 'download' button

How does it work?

- Git**Hub** is a platform for collaborative code development
- Transferring **local** repositories to a **remote** repository

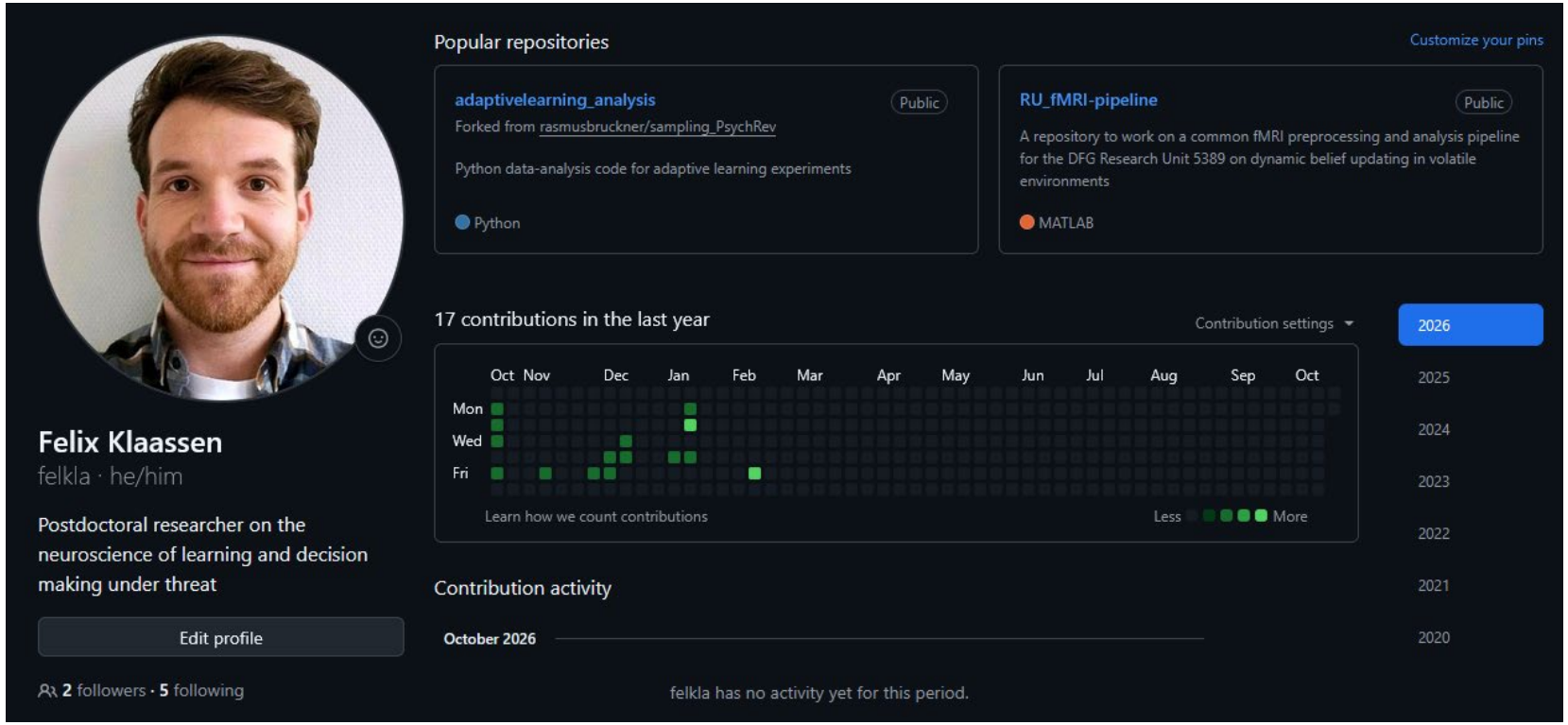


Basic workflow
(i.e., what you'll be doing for 95% of the time)

> Make some changes to your files
> '*git status*' check which files have changes
> '*git add*' changed files to the staging area
> '*git commit*' files from staging area to the db
> repeat
> ???
> Profit
> *'git push'* all changes from the **local** to the **remote** repository

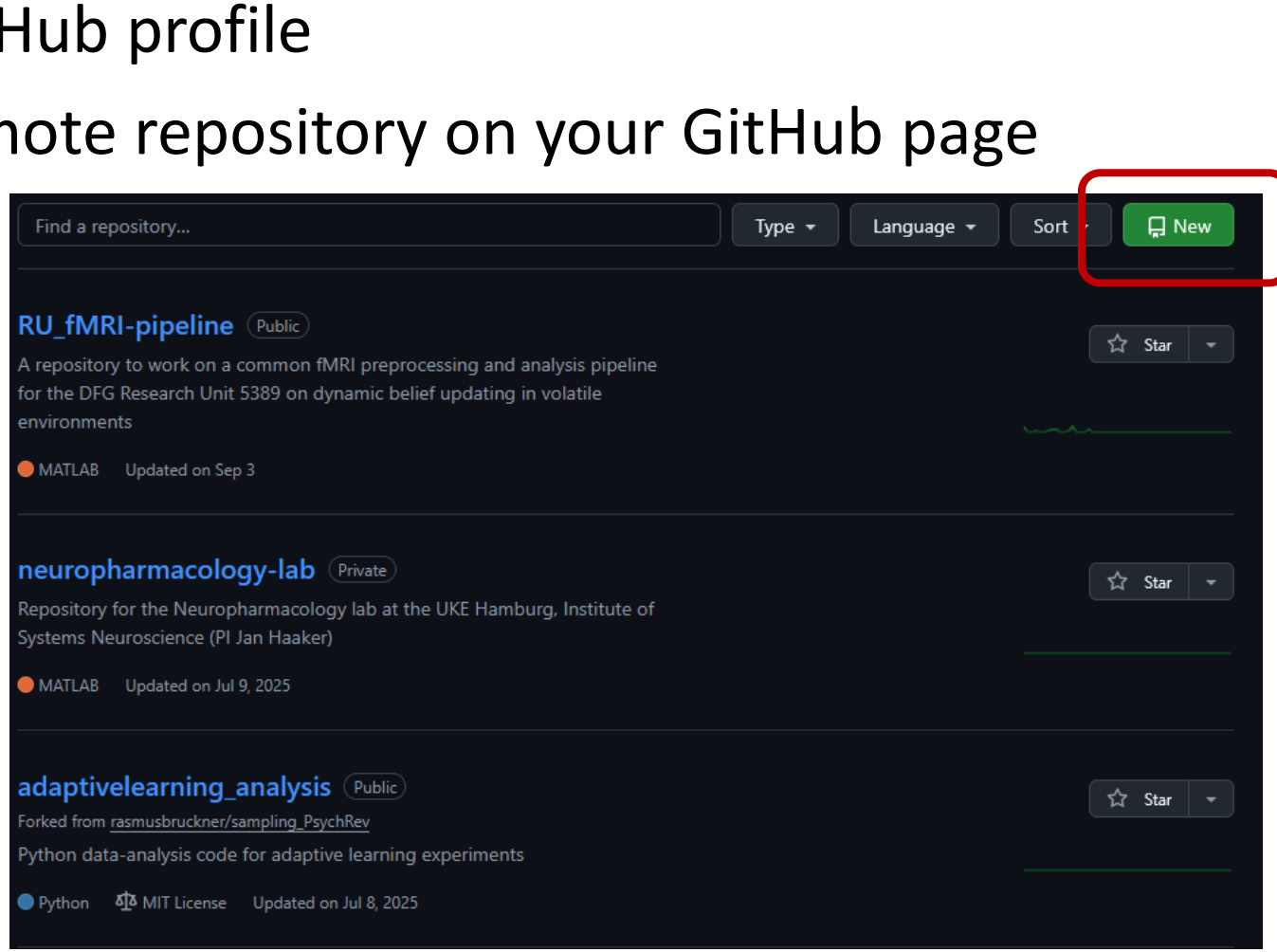
How do you use it?

- Create a GitHub profile



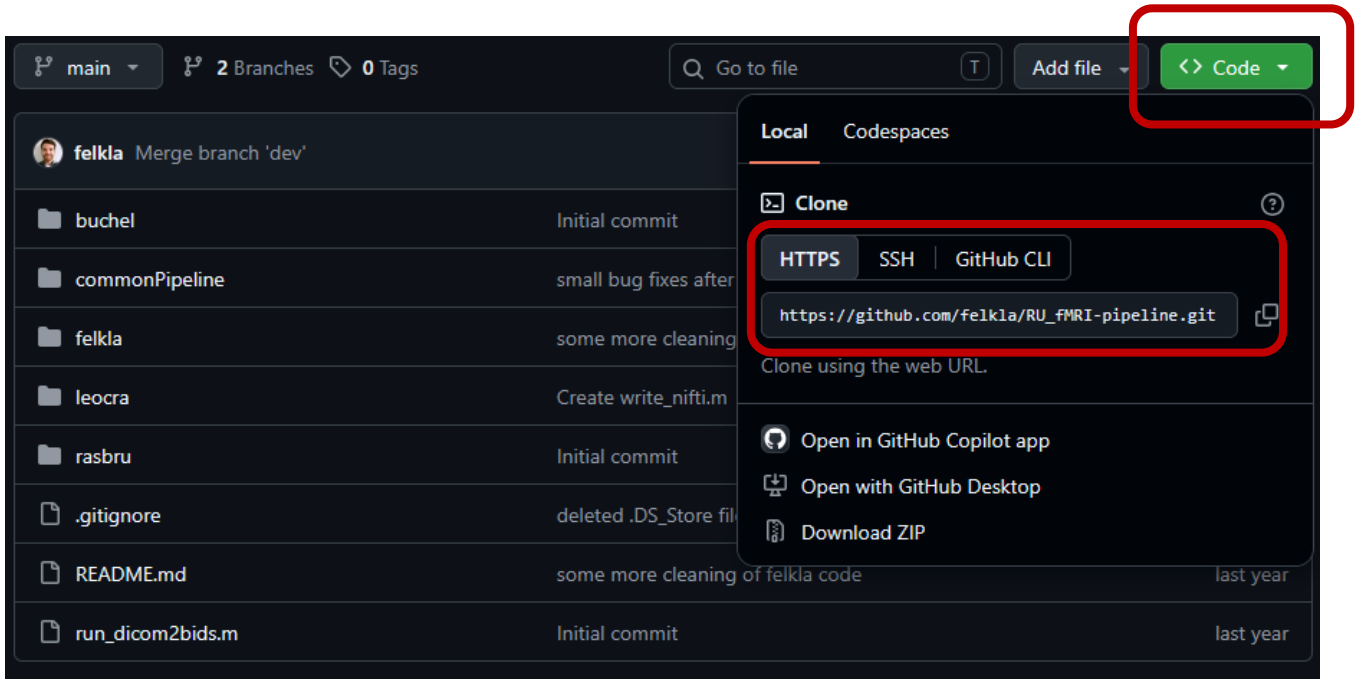
How do you use it?

- Create a GitHub profile
- Create a remote repository on your GitHub page



How do you use it?

- Create a GitHub profile
- Create a remote repository on your GitHub page
- Link your **local** repository to the **remote** repository



Open the Git Bash and navigate to your local repository
> git remote add origin <REMOTE-URL>
This links the local to the remote repository

> git push origin main
This 'pushes' the changes/files from your local to the remote repository (indicated by '*origin main*')

Referring to the remote repo ('*origin*' is the default name)

Referring to the branch '*main*'

Another tiny demo

https://github.com/felkla/RU_fMRI-pipeline

> gitk

Other IDEs: gitKraken

Some resources

- A tool to automatically generate a .gitignore file https://www.toptal.com/developers/gitignore
- An intuitive tutorial/introduction (used for these slides) https://julia-pfarr.github.io/git-gitlab_tutorial/
- gitKraken IDE to bypass the Git Bash: https://gitkraken.com/
- Generally: Google is your friend!

Thank you!

Questions? ☺